



COMBINED ERROR DETECTION AND ERROR CORRECTION

Prepared by:

- 1) Ahmed Taha Mohammed
- 2) Esraa Ahmed Ibrahim
- 3) Eman Mohammed Hussien
- 4) Zeyad Mostafa Abd El-Hafeez
- 5) Omar Waled Omar
- 6) Hana Sherif Abd El-Moneam

Under supervision of:

Dr. Mohammed Faysal

Signal Processing Project

1. Introduction

Modern communication systems are designed to send information correctly and efficiently. However, during transmission, signals can be affected by noise, interference, or fading. These problems may change some bits and cause errors in the received data.

To reduce these errors, communication systems use **error control coding**. This includes two main methods: **error detection** and **error correction**. **Error detection** only checks whether an error has happened. It does not fix the error. Instead, the receiver can ask the transmitter to send the data again. Common examples of error detection methods are parity checks, checksums, and cyclic redundancy checks (CRC). **Error correction** can both find and fix errors. It works by adding extra bits to the original data before transmission. These extra bits help the receiver recover the correct message even if some errors occur. Examples of error correction codes include Hamming codes, convolutional codes, turbo codes, LDPC codes, and block codes.

1.1. Analysis of Undetected Error Probability

Hamming distance laid the foundation for error-detecting and error-correcting codes in digital communications. By adding parity bits to create minimum distances between valid codewords, systems can detect when received data doesn't match valid patterns and even correct errors by mapping to the nearest valid codeword.

The mathematical foundation for combining error detection and correction relies on the properties of a linear code's minimum Hamming distance (d). A code with a minimum Hamming distance (d) can correct a maximum of $t = \lfloor \frac{d-1}{2} \rfloor$ errors. This value t defines a sphere of radius t centered at a codeword; if a received vector falls within this radius, it is decoded as that codeword.

However, a system does not always need to correct the maximum number of errors. It can choose a smaller correction radius, written as τ , where $\tau < t$. This means the system corrects fewer errors and uses the remaining ability to detect errors instead. If the received word is too far from any valid codeword, the system declares an error instead of making a wrong correction, effectively trading off the number of correctable errors to minimize the probability of an undetected error.

In geometric modeling of linear codes, a decoding sphere $S_q^\tau(C)$ is drawn around each valid codeword C . The radius τ represents the maximum number of errors the system is allowed to correct. If a received vector falls inside this sphere, the decoder assumes it was meant to be codeword C . To avoid confusion, decoding spheres around different codewords must not overlap. A smaller radius gives better error detection, while a larger radius corrects more errors but increases the chance of accepting a wrong codeword.

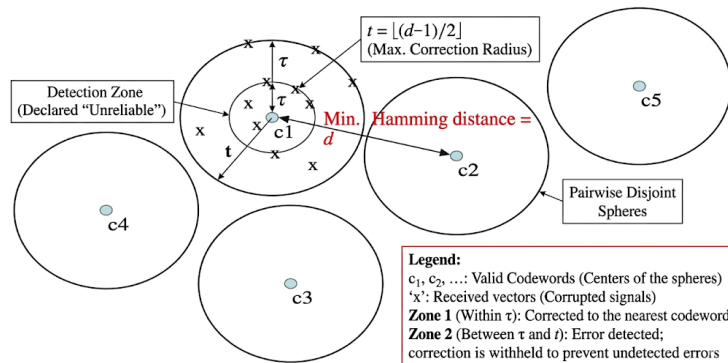


Fig1: Geometric representation of decoding spheres around codewords with minimum Hamming distance

An undetected error happens when channel noise alters a transmitted codeword so that it lands inside the decoding sphere of a completely different valid codeword. When the corrupted signal falls within the acceptable correction radius τ of this wrong codeword, the decoder is tricked into accepting it. Because the corrupted signal looks just like a valid message, the system outputs the wrong codeword without ever realizing a transmission error occurred.

To accurately determine the probability of an undetected error, the system must account for all possible corrupted vectors that could mistakenly fall within a decoding sphere.

Lemma 17.1

$$N_q^\tau(n, i, j) = \sum_{e=e_L}^{e_U} \sum_{\delta=\delta_L}^{\delta_U} \binom{i}{e} \binom{e}{\delta} \binom{n-i}{j-i+\delta} (q-1)^{j-i+\delta} (q-2)^{e-\delta}$$

provides the foundational mechanism to achieve this. Its core purpose is to mathematically count the exact number of vectors of a specific weight j that exist inside a sphere of radius τ . This exact count is essential for evaluating the statistical likelihood that a corrupted signal will be incorrectly accepted as a valid, unintended codeword.

2. Purpose of the project

The purpose of this project is to simulate the transmission of a color image through a noisy digital communication channel and study how different HARQ techniques improve reliability.

The image is first compressed into a JPEG-style bitstream. Then, the bitstream is transmitted using BPSK modulation over an AWGN noisy channel. At the receiver, different HARQ methods are used to detect or correct errors before reconstructing the image.

The project compares three main HARQ methods:

1. **Hard Decision CRC with Chase Combining HARQ**
2. **Incremental Redundancy HARQ with CRC**
3. **CRC-less Incremental Redundancy HARQ**

The main goal is to reduce bit errors, improve image reconstruction quality, and study the trade-off between reliability, retransmissions, and throughput.

3. Theory of Solution

The theory of the solution is based on **error control coding** in digital communication systems. During transmission, data can be corrupted by noise, interference, and fading. To solve this problem, communication systems use: Automatic Repeat reQuest (ARQ); It is an error-control method used in two-way digital communication. It works by adding a cyclic redundancy check (CRC) to each transmitted codeword. When the receiver checks the CRC, it can detect whether errors occurred during transmission. If an error is found, the receiver asks the transmitter to send the codeword again.

However, ARQ alone is not very efficient in noisy channels, such as wireless communication. Noise, interference, and multipath effects can cause many errors, which leads to frequent retransmissions and reduces system performance. Forward Error Correction (FEC) helps solve this problem by adding extra redundancy to the transmitted data. This allows the receiver to correct many errors without asking for retransmission. Common FEC methods include convolutional codes, turbo codes, LDPC codes, and block codes. However,

fixed-rate FEC can be inefficient because it always sends a large amount of redundancy, even when the channel is good. Also, FEC may not correct all errors.

Hybrid Automatic Repeat reQuest (HARQ) combines ARQ and FEC. First, FEC tries to correct errors at the receiver. Then, CRC checks whether any errors remain. If the CRC fails, the receiver requests another transmission or asks for extra parity information. This makes HARQ more reliable and efficient than using only ARQ or only FEC.

There are two main HARQ combining methods:

1. **Chase combining:** the transmitter sends the same packet again. The receiver combines repeated copies to improve the signal quality; using CRC to detect errors.
2. **Incremental redundancy:** the transmitter does not resend the same packet. Instead, it sends new parity bits, which give the receiver more information to decode the original data; using CRC to detect errors.

System	Incremental Redundancy HARQ with CRC	Incremental Redundancy (IR) HARQ Without a CRC
Using CRC	The transmitter adds CRC bits to the original data. Instead of sending the full message at once, it first sends only part of it: the data, CRC, and some parity bits.	The system does not use CRC bits. Instead, it uses a reliability test based on Euclidean distance.
At the receiver	At the receiver, the decoder tries to correct errors, then the CRC checks if the decoded data is correct.	The receiver finds the most likely codeword C_1 and the second most likely codeword C_2 . Then, it calculates the distance between the received signal and both codewords (Δ).
Advantages	High reliability.	Improves throughput because no CRC bits are transmitted.
Disadvantages	CRC bits reduce throughput because they are extra bits that do not carry useful data.	A higher threshold improves reliability but may cause more retransmissions.

Reliability estimation

It is the core mathematical engine that allows the CRC-less Incremental Redundancy (IR) HARQ system to function. Instead of relying on a simple binary pass/fail check like a CRC, it evaluates the soft distance between the received corrupted signal and the closest valid mathematical patterns.

When a packet of length n_1 symbols is received (denoted as vector y), the forward error-correction (FEC) decoder evaluates it and outputs the two most probable valid codewords:

- C_1 : The most likely transmitted codeword.
- C_2 : The next most likely transmitted codeword.

The estimator calculates the squared Euclidean distance between the actual received symbols y and both candidate codewords. This measurement dictates exactly how much noise separates the received signal from the valid patterns.

- For the primary candidate:

$$d_E^2(y, c_2) = \sum_{j=0}^{n_i-1} (y_j - c_{2j})^2$$

- For the secondary candidate:

$$d_E^2(y, c_1) = \sum_{j=0}^{n_i-1} (y_j - c_{1j})^2$$

To mathematically estimate how reliable C_1 is compared to C_2 , the system calculates the difference between these two distances, denoted as Δ :

$$\Delta = d_E^2(y, c_2) - d_E^2(y, c_1)$$

The size of Δ acts as a direct indicator of decoding confidence:

- If C_1 is correct: The distance $d_E^2(y, c_1)$ will be very small, representing only the actual squared noise samples added by the channel because the signal terms cancel out. Because the secondary codeword c_2 must differ from c_1 in at least a few positions equal to the code's minimum Hamming distance d_{min} , $d_E^2(y, c_2)$ will be significantly larger. Consequently, Δ will be a large number.
- If C_1 is incorrect: The received signal y is sitting somewhere in the ambiguous, noisy space between valid codewords. Both $d_E^2(y, c_1)$ and $d_E^2(y, c_2)$ will be relatively similar in size, resulting in a Δ that is very small.

Finally, Δ is passed to a comparator to be evaluated against a specific threshold. This threshold is calculated using three main factors:

- The minimum Hamming distance of the first code in the sequence.
 - The absolute noise power of the channel.
 - A tunable multiplicative constant, termed κ .
- ❖ If Δ is greater than or equal to this threshold, c_1 is declared reliable and correct, triggering an ACK.
 - ❖ If Δ is smaller than the threshold, c_1 is deemed insufficiently reliable, triggering a NACK for more parity symbols.

As the system requests more parity symbols, the d_{min} of the extended code increases, provided c_1 is correct, Δ will increase accordingly on the next decoding pass.

4. Methodology (Software Implementation)

4.1. JPEG image Encoding & Decoding

4.1.1. Purpose

- The main purpose of this code is to demonstrate how a color image can be compressed into a bitstream and reconstructed again using **JPEG compression technique**.
- The code covers the following:
 - 1) How an image is converted into data bits?
 - 2) How color images are separated into brightness and color components?
 - 3) How DCT transforms image pixels into frequency components?

- 4) How quantization reduces image detail?
- 5) How the image can be reconstructed from encoded bits?

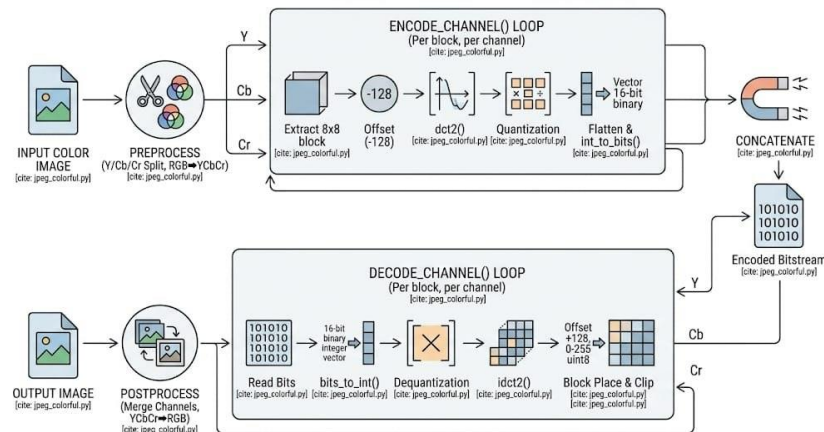


Fig2: Illustrative diagram showing the order of the main process of JPEG encoding

4.1.2. Software Implementation

4.1.2.1. Preprocessing the image

Before compression starts, the image must be prepared

- The image is first opened and converted to RGB format.

```
img = Image.open(image_path).convert('RGB')
```

- Since DCT is applied on **8×8 blocks**, the width and height of the image must be divisible by 8. If they are not, the image is cropped.

```
img_cropped = img.crop((0, 0, new_w, new_h))
```

4.1.2.2. Color Space Conversion

- The input image is first converted from **RGB** to **YCbCr**.

RGB stores color as:	YCbCr stores the image as:
1. R = Red channels	1. Y = brightness/luminance
2. G = Green channels	2. C _b = blue color difference
3. B = Blue channels	3. C _r = red color difference

```
img_ycbcr = img_cropped.convert('YCbCr')
```

This is useful because the human eye is **more sensitive to brightness changes** than color changes. Therefore, the code treats the brightness channel and color channels differently.

- **Y channel** = brightness / luminance

The eye is **more sensitive** to this, so the code keeps more detail using Q_MATRIX.

- **C_b and C_r channels** = color / chrominance

The eye is **less sensitive** to this, so the code compresses color more using C_MATRIX.

- Simple example:

If a picture becomes a little darker or brighter in one area, you may notice it quickly. But if the color changes slightly, especially in small details, you may not notice it as much.

- The YCbCr image is separated into three independent channels: Y, C_b, and C_r.

```
def encode_color_image_to_bits(img_ycbcr): #
    y_img, cb_img, cr_img = img_ycbcr.split()
```

4.1.2.3. Discrete Cosine Transform (DCT)

- The DCT converts image data from the **spatial domain** to the **frequency domain** because compression is easier in the frequency domain
 - Spatial domain means pixel values.
 - Frequency domain means how much low-frequency and high-frequency detail exists in the block.

```
def dct2(block): # define a function to apply 2D DCT to one 8x8 image block.
    """Apply 2D Discrete Cosine Transform (DCT) on an 8x8 block."""
    return dct(dct(block.T, norm='ortho').T, norm='ortho') # applies 2D DCT by transforming the block in both directions: rows and columns.
                                                           # .T swaps rows and columns
                                                           # norm='ortho' normalizes DCT to make reconstruction accurate by keeping the transform properly scaled.
```

- After DCT, each 8×8 block contains 64 frequency coefficients.
 - The top-left coefficient is called the **DC coefficient -low frequency component-**. It represents the average brightness of the block.
 - The other coefficients are called **AC coefficients -high frequency component-**. They represent changes, edges, texture, and fine details which are less important.

So; we use DCT because it helps us know which parts are important and which parts can be compressed

4.1.2.4. Quantization

- Quantization is the main compression step.
 - The code divides each DCT coefficient by a quantization matrix and rounds the result
- ```
quantized_block = np.round(dct_block / quant_matrix)
```
- Small values in the quantization matrix keep more detail.
    - The luminance matrix has smaller values because brightness is important to human vision.
  - Large values remove more detail.
    - The chrominance matrix has larger values because color details can be compressed more strongly.

This step causes information loss, which is why the reconstructed image may not be exactly identical to the original image.

#### 4.1.2.5. Encoding one channel

- This is the **encode\_channel** function

For each channel, the algorithm does the following steps:

1. Extract one 8x8 block from the channel.

```
block = channel_array[i:i + 8, j:j + 8]
```
2. Centers pixel values by subtracting 128.
  - Before applying DCT, the code subtracts 128 from every pixel value
  - Pixel values normally range from: 0 to 255; after subtracting 128, they are centered around zero: approximately -128 to +127

This helps the DCT work more effectively because the signal becomes centered around zero.

```
centered_block = block.astype(np.float64) - 128.0
```

3. Applies DCT.

```
dct_block = dct2(centered_block)
```
4. Quantizes the DCT coefficients.

```
quantized_block = np.round(dct_block / quant_matrix)
```
5. Flattens the 8×8 matrix into a 1D list.

```
flattened = quantized_block.flatten()
```

6. Converts each coefficient into binary bit and adds the bits to the final bitstream.

```
for coeff in flattened: # Loop through every quantized coefficient.
 bits = int_to_bits(coeff, num_bits=16) # Convert the coefficient into 16 bits.
 bitstream.extend(bits) # Add the 16 bits to the total bitstream.
```

#### int\_to\_bits function:

```
format_str = f'{{:0(num_bits)b}}' # creates a format rule to write any number as binary using exactly num_bits bits, adding zeros at the beginning if needed.
binary_str = format_str.format(int(value) & ((1 << num_bits) - 1)) # converts the coefficient to binary and uses num_bits bits to represent both positive and negative values.
shift <<: moves 1 left by num_bits places to create the bit range size.
subtract - 1: changes that size into a mask made of num_bits ones.
AND &: applies the mask to keep only the needed num_bits bits.
```

#### 4.1.2.6. Encoding the full color image

After creating the encoder for a single channel, the program applies it to all three YCbCr channels.

```
def encode_color_image_to_bits(img_ycbcr): # Encodes the full color image into one bitstream.
```

- 1) The YCbCr image is split into its three channels.
- 2) Each channel is encoded separately using the `encode_channel()` function.
- 3) The Y channel uses the luminance quantization matrix, while the Cb and Cr channels use the chrominance quantization matrix.
- 4) Finally, the three bitstreams are concatenated into one complete bitstream.

```
return np.concatenate([y_bits, cb_bits, cr_bits])
```

#### 4.1.2.7. Decoding one channel

- The `decode_channel` function reverses the encoding process:

1. Reads 16 bits at a time.
2. Converts bits back into coefficients.
3. Rebuilds the 8×8 coefficient block.

```
for x in range(8): # Loop through the rows of the 8x8 coefficient block.
 for y in range(8): # Loop through the columns of the 8x8 coefficient block.

 bits = bitstream[bit_idx:bit_idx + 16] # Read the next 16 bits from the bitstream.
 bit_idx += 16 # Move the bit index forward by 16 bits.
 block_coeffs[x, y] = bits_to_int(bits, num_bits=16) # Convert bits back into a signed coefficient.
```

4. Dequantizes the block.
5. Applies inverse DCT.
6. Adds 128 back.
7. Clips pixel values to the range 0–255.

```
dequantized_block = block_coeffs * quant_matrix # Reverse quantization by multiplying by the quantization matrix.
idct_block = idct2(dequantized_block) # Apply inverse DCT to return to spatial-domain pixels.
reconstructed_block = idct_block + 128.0 # Add 128 to reverse the centering step.
reconstructed_channel[i:i + 8, j:j + 8] = reconstructed_block # Insert the reconstructed block into the channel.

return np.clip(reconstructed_channel, 0, 255).astype(np.uint8) # Clip values to 0-255 and convert to image pixel format.
```

#### 4.1.2.8. Decoding the full color image

- The full bitstream contains the Y bits first, then the Cb bits, then the Cr bits. The decoder calculates how many bits belong to each channel and separates them.

```
def decode_bits_to_color_image(bitstream, h, w):
```

- . After reconstruction, the three channels are converted back into image format and merged into one YCbCr image.

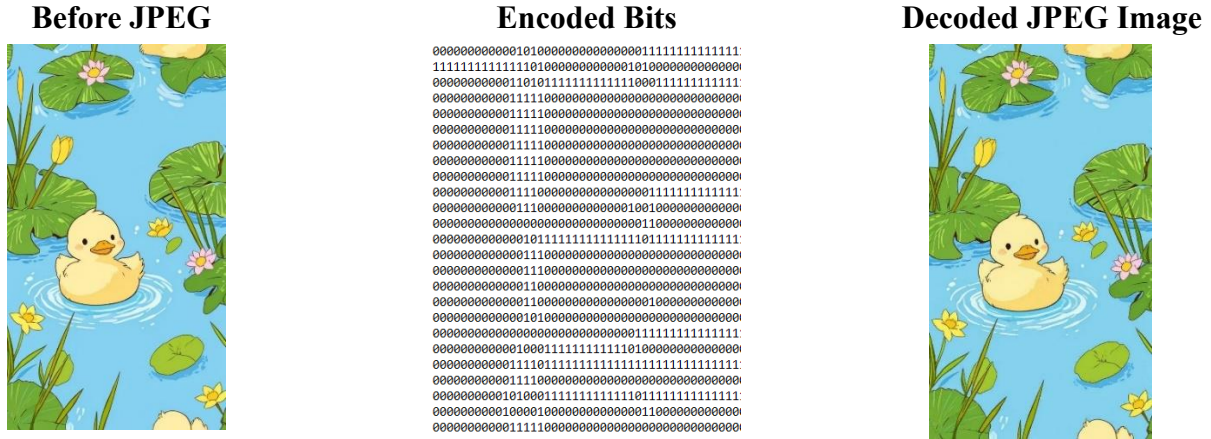
```
return final_ycbcr # Return reconstructed YCbCr image.
```

#### 4.1.2.9. Final reconstruction and saving the image

- After the bitstream is decoded, the reconstructed image is obtained in YCbCr format. It is then converted back to RGB format so it can be displayed and saved as a normal color image. The final reconstructed image is saved as a JPEG file.

```
final_rgb = final_ycbcr.convert('RGB')
```

#### 4.1.3. Results



### 4.2. Hard-Decision CRC + Chase Combining HARQ

#### 4.2.1. Purpose

The purpose of this method is to simulate the transmission of a color image through a noisy digital communication channel using a Hard-Decision CRC-based Chase Combining HARQ system. The image is first converted into a JPEG-style bitstream. Each block is protected using an 8-bit CRC, which allows the receiver to detect whether errors occurred during transmission.

At the receiver side, hard-decision decoding is applied to convert the noisy received symbols back into binary bits. The CRC is then checked to determine whether the block was received correctly.

If the CRC check passes, the block is accepted and an ACK is generated. If the CRC check fails, the receiver requests retransmission using a NACK. The system allows multiple retransmissions for each block, and the received copies are combined “this is the **Chase Combining**” to improve the chance of correct decoding. After all blocks are processed, the received bitstream is decoded back into a color image and saved as the reconstructed image.

#### 4.2.2. Software Implementation

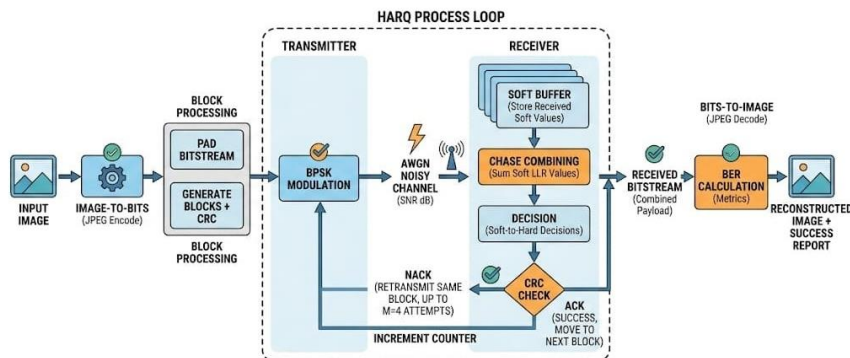


Fig3: Illustrative diagram showing the order of the main process of Hard Decision CRC + CC HARQ

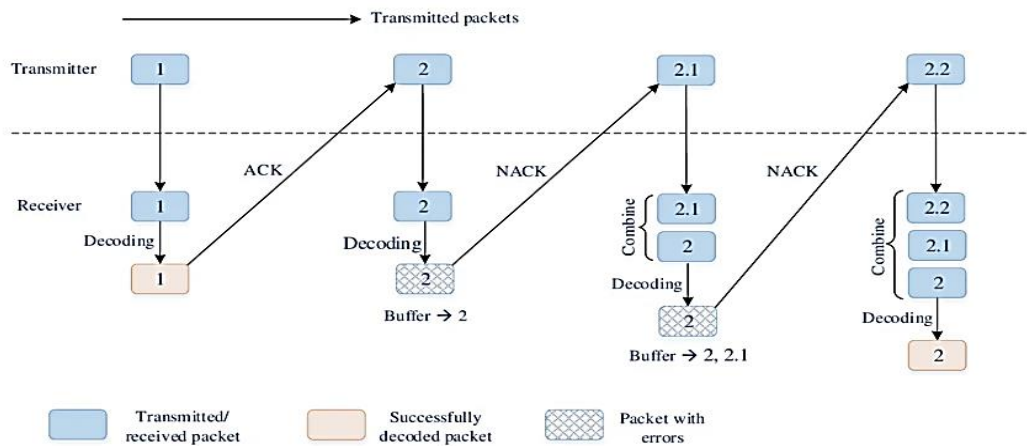


Fig4: Illustrative diagram showing the order of the main process of HARQ with chase combining

#### 4.2.2.1. CRC Encoding

##### 1. Choose the divisor

- In CRC, both sender and receiver must use the same generator polynomial, also called the divisor.
- CRC8\_POLY = 0x2F is a CRC-8 polynomial; CRC size = 8 bits, Divisor size = 8 + 1 = 9 bits
- Full divisor has an invisible leading 1; CRC8\_POLY = 0010 111; Full divisor = 1 0010 1111
- So mathematically, the divisor is  $P(x) = x^8 + x^5 + x^3 + x^2 + x + 1$
- initialize the remainder register that stores the temporary remainder during division.

```
CRC8_POLY = 0x2F
def crc_encode(u):
 reg = 0 # initialize the CRC register to zero "remainder"
```

##### 2. Start the division process

- it uses a shift register to perform the division; Dataword ÷ Divisor
- checks the leftmost bit of the 8-bit CRC register; For CRC-8, the leftmost bit is bit number 7

```
for bit in u.astype(np.uint8): # read each data bit one by one.
 msb = (reg >> 7) & 1 # get the most significant bit of the CRC register by shifting it 7 places to the right.
 # & 1: keeps only the last bit.
 # Save the leftmost bit of the CRC register before changing the register.
```

- Bring down the next data bit

```
reg = ((reg << 1) & 0xFF) | int(bit)
```

It does three things:

- `reg << 1`: Shifts the register left by one position. Mathematically, this is like multiplying by  $x$ . In normal binary number language, `reg << 1` means multiply by 2. In CRC polynomial language, `reg << 1` means multiply by  $x$ . Mathematically, that becomes:  $x(x^3 + x + 1) = x^4 + x^2 + x$
- `& 0xFF`: Keeps only 8 bits because this is CRC-8.
- `| int(bit)`: Places the new input bit into the rightmost position.

- XOR with divisor if needed

In CRC, subtraction is done using XOR because CRC works in binary arithmetic, also called GF(2).

- If leftmost bit is 1 → XOR with divisor
- If leftmost bit is 0 → XOR with zeros

```
if msb: # If the leftmost significant bit was 1
 reg ^= CRC8_POLY # apply XOR with the CRC polynomial "Divisor".
```

### 3. Append zeros to the dataword

- In CRC, before finding the final remainder, we append zeros to the dataword.
- The number of zeros is: Number of divisor bits – 1; the divisor has 9 bits, so:  $9 - 1 = 8$  zeros  
Augmented dataword = original dataword + 00000000
- Continue division using the appended zeros

```
for _ in range(8): # add 8 zero bits to finish the CRC calculation.
 msb = (reg >> 7) & 1
 reg = (reg << 1) & 0xFF

 if msb:
 reg ^= CRC8_POLY
```

This is the same division process as before, but now the input bits are zeros.

- After this loop finishes, the value inside reg is the final CRC remainder.

```
crc_bits = np.array([(reg >> i) & 1 for i in range(7, -1, -1)], dtype=np.uint8) # Convert the CRC value into 8 bits: the loop start at 7, stop before -1, move by -1 each time

return np.concatenate([u.astype(np.uint8), crc_bits]) # Return the original data bits followed by CRC bits; Attach the remainder to the dataword to form the codeword
```

Codeword = Dataword + CRC

#### 4.2.2.2. Checking Block

- The function `crc_check()` is used at the receiver side. It separates the received block into: received data bits + received CRC bits

```
def crc_check(x_hat): # Checks if a received block passes the CRC test.
 u_hat = x_hat[:-8].astype(np.uint8) # Take all bits except the last 8 bits as data.
 crc_hat = x_hat[-8:].astype(np.uint8) # Take the last 8 bits as the received CRC.
```

- Then it recalculates the CRC from the received data bits and compares it with the received CRC.
  - If both are equal, the block is assumed to be correct and the receiver sends an **ACK**.
  - If they are different, the block is assumed to contain errors and the receiver sends a **NACK**.

```
recomputed_crc = crc_encode(u_hat)[-8:] # Recalculate the CRC from the received data bits.

passed = np.array_equal(crc_hat, recomputed_crc) # Compare received CRC with recalculated CRC.

return passed, u_hat # Return CRC result and the received data bits without CRC.
```

#### 4.2.2.3. BPSK Modulation and AWGN Channel Block

This block represents the **digital communication channel**.

- The function `bpsk_mod()` converts binary bits into BPSK symbols:
  - bit 0 → +1
  - bit 1 → -1
- After modulation, the signal is passed through an **AWGN channel** using the function `awgn_channel()`. AWGN stands for **Additive White Gaussian Noise**, which simulates the noisy transmission environment.
- The noise power depends on the value of  $E_b/N_0$ . A higher  $E_b/N_0$  means a cleaner channel, while a lower  $E_b/N_0$  means more noise and more bit errors.

#### 4.2.2.4. Hard-Decision + CC HARQ Transmission Block

- Its algorithm is implemented in `harq_decision_harq_for_bitstream` function.
- First, the input image bitstream is divided into blocks of size  $k = 1024$  bits. If the final block is shorter than 1024 bits, zero padding is added so that all blocks have the same size.
- For each block, CRC bits are added before transmission.

```
for block_index in range(total_blocks): # Process each block separately.
 u = input_bits[block_index * k:(block_index + 1) * k] # Extract one block of data bits.
 x = crc_encode(u) # Add CRC bits to the block.
 code_rate = k / len(x) # Calculate the effective code rate after adding CRC bits.
```

- The block is then modulated using BPSK and sent through the AWGN channel.
- The receiver uses **hard-decision decoding**:
  - First, **Chase Combining**: combine all received versions of the same block before making the final hard decision. This improves the reliability of the received signal because repeated transmissions are added together, reducing the effect of random noise.
 

```
this is the key of being HYBRID
combined_soft = np.sum(np.vstack(rx_buffer), axis=0)
```
  - received value  $< 0 \rightarrow$  bit 1
  - received value  $\geq 0 \rightarrow$  bit 0
 

```
x_hat = (combined_soft < 0).astype(np.uint8)
```
- check if the decoded block passes CRC.
 

```
passed, u_hat = crc_check(x_hat)
last_u_hat = u_hat
```

  - if passed:
 

```
received_payload_bits.extend(u_hat) # Add the correct received data bits to the output.
ack_blocks += 1 # Increase the ACK counter.
block_passed = True # Mark this block as successfully received.
```
  - if not passed:
 

```
received_payload_bits.extend(last_u_hat) # Use the last decoded version even though it failed CRC.
nack_blocks += 1 # Increase the NACK counter.
```
- Finally, convert the received bits list to a NumPy array and remove padding bits and restore the original length.
 

```
received_payload_bits = np.array(received_payload_bits, dtype=np.uint8)
received_payload_bits = received_payload_bits[:original_length]
```

### 4.2.3. Results

#### CRC pass after retransmission

```
===== Block 10/36603 =====
Transmission 1: CRC check: FAIL
NACK: retransmission requested.
CRC check: PASS
ACK: decoded correctly after 2 transmissions.
Final decision: ACK
```

#### CRC fails

```
===== Block 2015/36603 =====
Transmission 1: CRC check: FAIL
NACK: retransmission requested.
Transmission 2: CRC check: FAIL
NACK: retransmission requested.
Transmission 3: CRC check: FAIL
NACK: retransmission requested.
CRC check: FAIL after 3 transmissions.
Final decision: NACK
```

#### System Characteristics

```
===== Hard-Decision CRC + HARQ SUMMARY =====
Total blocks: 36603
ACK blocks: 36593
NACK blocks: 10

Bit errors after H-D CRC + HARQ: 402
BER after H-D CRC + HARQ: 0.00001073
```

**Image before transmission**



**Image after transmission with Hard Decision CRC + CC HARQ**



### 4.3. Incremental Redundancy (IR) HARQ Without a CRC

#### 4.3.1. Purpose

The purpose of this code is to simulate CRC-less IR-HARQ image transmission over a noisy channel. It uses reliability estimation instead of CRC to decide whether to accept a received block or request more redundancy, then reconstructs the received image and calculates BER.

#### 4.3.2. Software Implementation

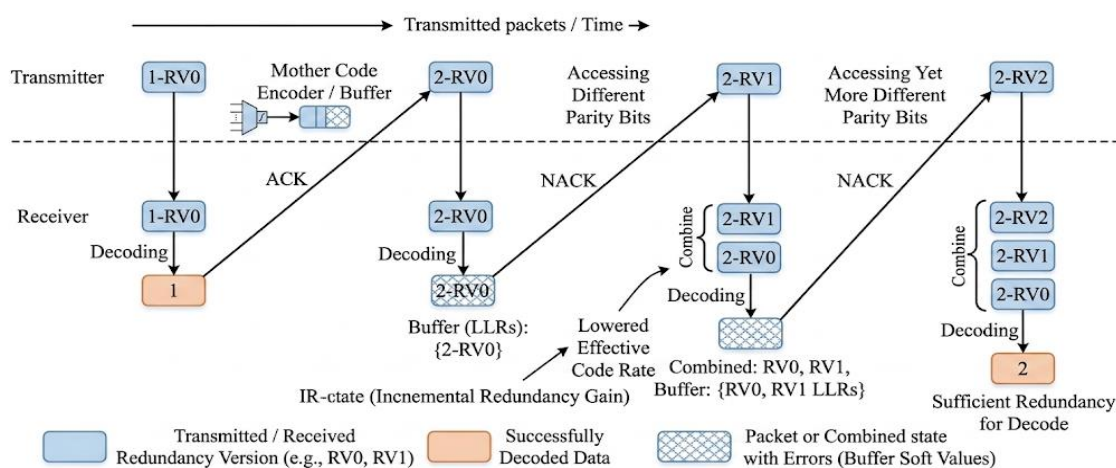


Fig5: Illustrative diagram showing the order of the main process of HARQ with Incremental redundancy

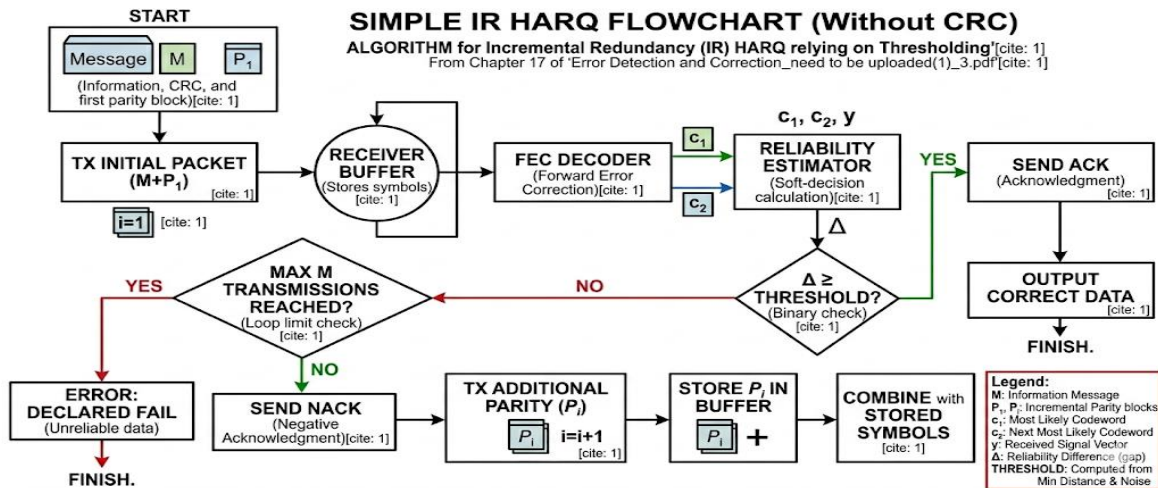


Fig6: Algorithm IR without CRC HARQ Flowchart

#### 4.3.2.1. Main CRC-less IR-HARQ Function

##### 1. Mother Code Generation

- For each block, the transmitter creates several redundancy versions. These are called **RVs**, or **Redundancy Versions**.
- The first redundancy version is: original data block
- The other redundancy versions are generated by XORing the original block with shifted versions of itself.
- These extra versions act like parity information. They help the receiver improve its decision if the first transmission is not reliable.
- The redundancy versions are stored in a list: `mother_code_rvs = [rv0, rv1, rv2, rv3]`

```

for block_index in range(total_blocks):
 u = input_bits[block_index * k:(block_index + 1) * k] # Extract the exact 1024 bits for this block

 # MOTHER CODE GENERATION (Creating Math Equations from Data)
 rv0 = u # RV0: The raw data itself (First transmission)
 rv1 = np.bitwise_xor(u, np.roll(u, 1)) # RV1: Shift data right by 1, XOR with original (Mathematical Link 1)
 rv2 = np.bitwise_xor(u, np.roll(u, 3)) # RV2: Shift data right by 3, XOR with original (Mathematical Link 2)
 rv3 = np.bitwise_xor(u, np.roll(u, 5)) # RV3: Shift data right by 5, XOR with original (Mathematical Link 3)

 mother_code_rvs = [rv0, rv1, rv2, rv3] # Group them all up. Transmitter holds this in memory.

```

##### 2. Receiver Soft Buffer

- The receiver creates a soft buffer:
- This buffer stores the noisy received symbols for each transmission attempt.
- Since  $M = 4$ , the buffer has four rows: row 0 → received RV0  
row 1 → received RV1  
row 2 → received RV2  
row 3 → received RV3
- The buffer is called a **soft buffer** because it stores real-valued noisy symbols, not only hard bits.
- For example, instead of storing only: 0 or 1; it stores values such as: +0.9, -1.4, +0.2, -0.7; These values contain reliability information.

```

rx_buffer_soft = np.zeros((M, rv_size), dtype=np.float64) # Receiver memory: 4 rows of 1024 empty float slots
block_passed = False
last_c1_bits = None # Saves the best guess just in case we fail all 4 attempts

```

### 3. Transmission Loop

- For each block, the code allows up to M transmissions; the current redundancy version is selected using:

```
for tx_num in range(M):
 tx_bits = mother_code_rvs[tx_num]
```

So: Transmission 1  $\rightarrow$  RV0

Transmission 2  $\rightarrow$  RV1

- Each selected redundancy version is then modulated using BPSK; Then, it is sent through the noisy AWGN channel

```
tx_symbols = bpsk_mod(tx_bits) # Convert 0s and 1s to +1.0 and -1.0 volts
rx_symbols = awgn_channel(tx_symbols, ebn0_db, code_rate=1.0, rng=rng) # Send through noisy air
```

- The received noisy symbols are stored in the soft buffer

```
rx_buffer_soft[tx_num, :] = rx_symbols
```

### 4. Soft Combining

- After receiving one or more transmissions, the receiver combines the available information.
- The combining starts with the first received transmission; If a second transmission is available, the receiver adds information from RV1 and so on.

```
1. Soft Combining (Merge the broken pieces together to cancel noise)
combined_energy = rx_buffer_soft[0, :].copy() # Start with RV0 energy as base
if tx_num >= 1:
 combined_energy += rx_buffer_soft[1, :] * np.roll(rx_buffer_soft[0, :], 1)
if tx_num >= 2:
 combined_energy += rx_buffer_soft[2, :] * np.roll(rx_buffer_soft[0, :], 3)
if tx_num >= 3:
 combined_energy += rx_buffer_soft[3, :] * np.roll(rx_buffer_soft[0, :], 5)
```

- The purpose of soft combining is to improve the reliability of the decoded bits by using all received transmissions together.

### 5. Finding the Most Likely Codeword $c_1$

- After soft combining, the receiver makes a hard decision:

```
c1_bits = (combined_energy < 0).astype(np.uint8)
```

The decision rule is: combined energy  $\geq 0 \rightarrow$  bit 0

combined energy  $< 0 \rightarrow$  bit 1

- So c1\_bits is the receiver's best estimate of the original block; This candidate is called: **c1 = most likely codeword**

### 6. Finding the Weakest Bit

- The receiver then finds the bit with the lowest reliability:

```
weakest_bit_idx = np.argmin(np.abs(combined_energy))
```

- The value abs(combined\_energy) shows how confident the receiver is.
- For example: combined\_energy = +2.5  $\rightarrow$  strong confidence for bit 0  
combined\_energy = -2.1  $\rightarrow$  strong confidence for bit 1  
combined\_energy = +0.05  $\rightarrow$  weak confidence for bit 0

### 7. Creating the Second Most Likely Codeword $c_2$

- After finding the weakest bit, the receiver creates a second candidate:

```
c2_bits = c1_bits.copy() # Copy the best guess
c2_bits[weakest_bit_idx] ^= 1 # Flip that one uncertain bit (0 becomes 1, 1 becomes 0) to make our second guess
```

- This means the receiver copies `c1_bits` and flips the weakest bit.
- So: `c1` = best guess  
`c2` = second-best guess
- The purpose of `c2` is to test how reliable `c1` really is.
  - ❖ If `c1` and `c2` are very close in reliability, the receiver should not trust `c1`.
  - ❖ But if `c1` is much better than `c2`, the receiver can accept `c1`.

## 8. Re-encoding $c_1$ and $c_2$

- The receiver re-encodes both candidates:

```
c1_rvs = [
 c1_bits,
 np.bitwise_xor(c1_bits, np.roll(c1_bits, 1)),
 np.bitwise_xor(c1_bits, np.roll(c1_bits, 3)),
 np.bitwise_xor(c1_bits, np.roll(c1_bits, 5))
]
```

This creates a parity version by XORing `c1_bits` with a shifted version of itself; creates another parity version using a shift of 3 positions & 5 positions.

- This step creates the redundancy versions that would have been transmitted if `c1` or `c2` were the correct block.
- The purpose is to compare actual received noisy signal with ideal signal if `c1` was correct and ideal signal if `c2` was correct; This allows the receiver to measure which candidate better matches the received signal.

## 9. Squared Euclidean Distance

- The receiver calculates the squared Euclidean distance for both candidates:

```
for i in range(tx_num + 1): # Only compare against the transmissions we have actually received so far
 y_i = rx_buffer_soft[i, :] # Grab actual noisy signal from memory
 dE2_c1 += np.sum((y_i - bpsk_mod(c1_rvs[i]))**2) # Distance formula: sum of (Actual - Perfect Guess 1)^2
 dE2_c2 += np.sum((y_i - bpsk_mod(c2_rvs[i]))**2) # Distance formula: sum of (Actual - Perfect Guess 2)^2
```

- The squared Euclidean distance is:  $d^2(y, c) = \sum (y_i - c_i)^2$   
 where:  $y_i$  is the received noisy signal,  $c_i$  is the ideal signal from candidate codeword
- smaller distance → better candidate “closer to the received signal”  
 larger distance → worse candidate

## 10. Reliability measure

- After calculating both distances, the code calculates the delta  
`delta = dE2_c2 - dE2_c1`
- It measures how much better the best candidate  $c_1$  is compared to the second-best candidate  $c_2$ .
  - If delta is large: `c1` is much better than `c2`; receiver is confident
  - If delta is small: `c1` and `c2` are very close; receiver is not confident

## 11. Threshold-Based ACK/NACK Decision

- Since this is a CRC-less system, the receiver does not use CRC to decide whether the block is correct.
- Instead, it uses the threshold rule:
  - $\text{delta} \geq \text{threshold} \rightarrow \text{reliability is high} \rightarrow \text{ACK}$
  - $\text{delta} < \text{threshold} \rightarrow \text{reliability is low} \rightarrow \text{NACK}$

## 12. ACK Case

- If the block passes the threshold test, the receiver accepts `c1_bits`:  

```
received_payload_bits.extend(c1_bits) # Add the good bits to our final output list
ack_blocks += 1
block_passed = True
```
- This means:
  - The receiver trusts the decoded block.
  - The block is counted as ACK.
  - No more transmissions are requested for this block.

## 13. NACK and Retransmission Case

- If the confidence gap is below the threshold, the receiver does not trust the decoded block
- In this case, the receiver requests another transmission.

### 4.3.3. Results

#### ACK after retransmission

```
===== Block 10/36603 =====
Transmission 1:
-> Evaluated c1 (Most Likely) vs c2 (Next Most Likely)
-> dE2(y, c1) = 123.67
-> dE2(y, c2) = 123.76
-> Delta = 0.10 | Threshold = 4.00
-> Result: Delta < Threshold. Reliability is LOW.
-> NACK: retransmission requested.
Transmission 2:
-> Evaluated c1 (Most Likely) vs c2 (Next Most Likely)
-> dE2(y, c1) = 246.48
-> dE2(y, c2) = 255.41
-> Delta = 8.93 | Threshold = 4.00
-> Result: Delta >= Threshold. Reliability is HIGH.
-> ACK: decoded correctly after 2 transmissions.
Final decision: ACK
```

#### NACK

```
===== Block 7287/36603 =====
Transmission 1:
-> Evaluated c1 (Most Likely) vs c2 (Next Most Likely)
-> dE2(y, c1) = 127.32
-> dE2(y, c2) = 127.34
-> Delta = 0.02 | Threshold = 4.00
-> Result: Delta < Threshold. Reliability is LOW.
-> NACK: retransmission requested.
Transmission 2:
-> Evaluated c1 (Most Likely) vs c2 (Next Most Likely)
-> dE2(y, c1) = 261.36
-> dE2(y, c2) = 256.68
-> Delta = -4.68 | Threshold = 4.00
-> Result: Delta < Threshold. Reliability is LOW.
-> NACK: retransmission requested.
Transmission 3:
-> Evaluated c1 (Most Likely) vs c2 (Next Most Likely)
-> dE2(y, c1) = 387.62
-> dE2(y, c2) = 378.46
-> Delta = -9.17 | Threshold = 4.00
-> Result: Delta < Threshold. Reliability is LOW.
-> NACK: retransmission requested.
Transmission 4:
-> Evaluated c1 (Most Likely) vs c2 (Next Most Likely)
-> dE2(y, c1) = 515.89
-> dE2(y, c2) = 503.53
-> Delta = -12.36 | Threshold = 4.00
-> Result: Delta < Threshold. Reliability is LOW.
-> NACK: retransmission requested.
Final decision: NACK
```

## System Characteristics

```
===== CRC-less IR-HARQ SUMMARY =====
Total blocks: 36603
ACK blocks: 36602
NACK blocks: 1
```

```
Bit errors after CRC-less IR-HARQ: 368
BER after CRC-less IR-HARQ: 0.00000982
```

**Image before transmission**



**Image after transmission with  
CRC-less IR HARQ**

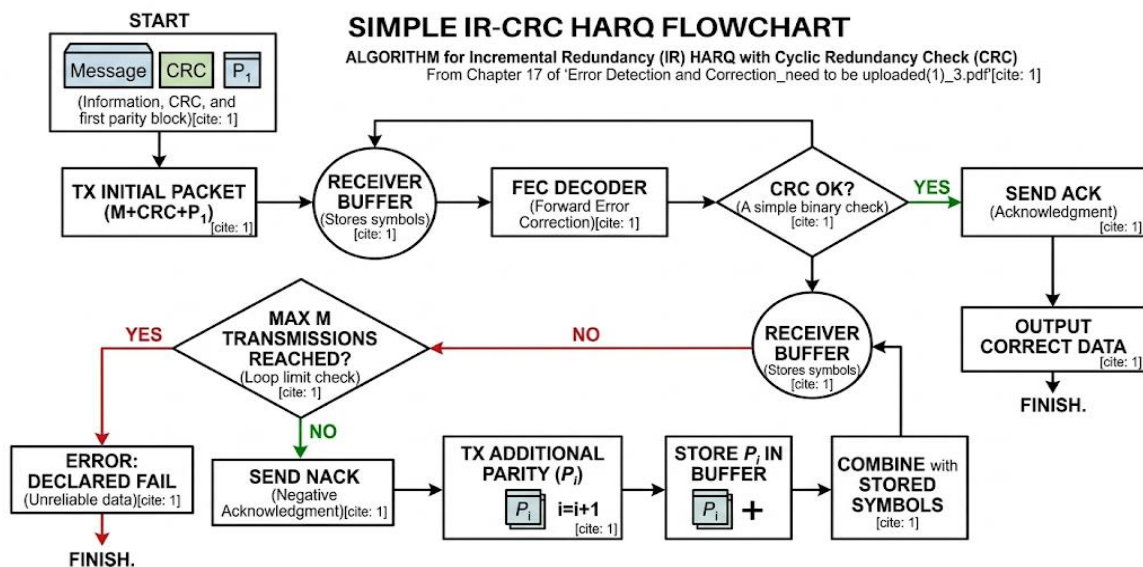


#### 4.4. Incremental Redundancy HARQ with CRC

##### 4.4.1. Purpose

The purpose of this code is to simulate the transmission of a compressed color image through a noisy channel using CRC to detect errors and IR-HARQ to request retransmissions when errors occur. Finally, it reconstructs the received bits back into an image and calculates the bit error rate to evaluate the transmission performance.

##### 4.4.2. Software implementation



- The image is first converted into a bitstream.
- The bitstream is split into blocks of 1024 bits.

- Each block is protected using an 8-bit CRC.
- The transmitter generates multiple redundancy versions using XOR operations and circular shifts.
- The first redundancy version is transmitted using BPSK modulation.
- The AWGN channel adds Gaussian noise to the transmitted signal.
- The receiver stores the noisy symbols in a soft buffer.
- After each transmission, the receiver combines the available received versions.
- The combined signal is converted back into bits using hard decision.
- The CRC check decides whether the block is accepted or retransmitted.
- If the CRC passes, an ACK is sent.
- If the CRC fails, a NACK is sent and another redundancy version is transmitted.
- After all blocks are processed, the received bitstream is compared with the original one to calculate BER.
- Finally, the received bitstream is decoded back into an image.

#### 4.4.3 Results

##### CRC passes after retransmission

```

===== Block 10/36603 =====
Transmission 1: CRC check: FAIL
NACK: retransmission requested.
CRC check: PASS
ACK: decoded correctly after 2 transmissions.
Final decision: ACK

```

##### CRC fails

```

===== Block 7236/36603 =====
Transmission 1: CRC check: FAIL
NACK: retransmission requested.
Transmission 2: CRC check: FAIL
NACK: retransmission requested.
Transmission 3: CRC check: FAIL
NACK: retransmission requested.
Transmission 4: CRC check: FAIL
NACK: retransmission requested.
CRC check: FAIL after 4 transmissions.
Final decision: NACK

```

##### System Characteristics

```

===== True IR-HARQ SUMMARY =====
Total blocks: 36603
ACK blocks: 36602
NACK blocks: 1

Bit errors after True IR-HARQ: 413
BER after True IR-HARQ: 0.00001102

```

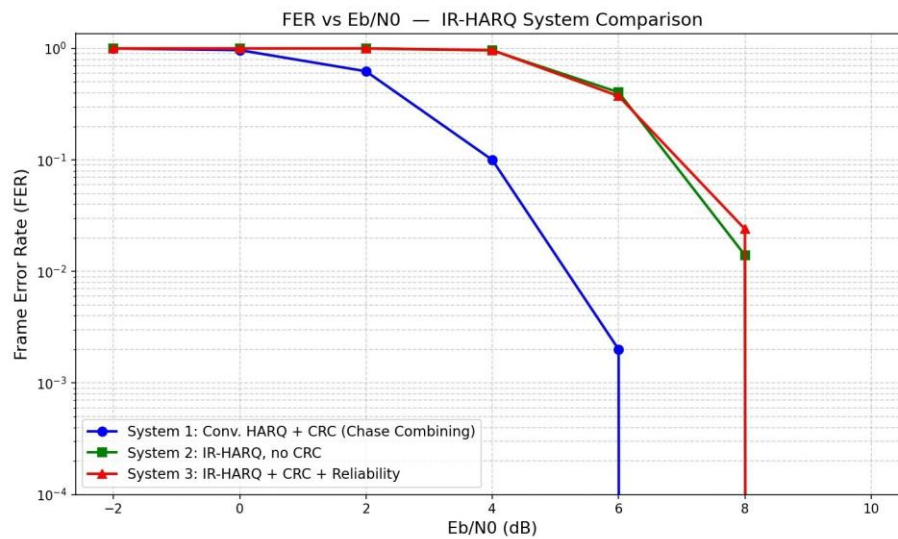
**Image before transmission**



**Image after transmission with  
IR HARQ with CRC**



## 5. Performance Results



- The frame error rate (FER) decreases as  $E_b/N_0$  increases.
- This means that all systems perform better when the channel quality improves.
- **System 1:** gives the best performance among the three systems.  
shows a fast drop in FER and reaches a very low error rate around 6 dB.
- **Systems 2 and 3:** have poor performance at low  $E_b/N_0$ , where the FER remains close to 1.  
start to improve noticeably only at higher  $E_b/N_0$ , especially around 6–8 dB.
- **System 3:** performs slightly worse than System 2 at high  $E_b/N_0$ .